

ZeroDrift — The Complete Build & Operations Guide

One page that covers everything the team can build, run, regenerate and deploy. Every command is copy-paste. (Windows commands use `\`, Mac/Linux `/`.)

1. Get the code

```
git clone https://github.com/pranshukesavmishra/SIH-2026.git
cd SIH-2026
```

2. Run the software from source (any laptop, 5 minutes)

Needs Python 3.10+ (<https://python.org> , tick "Add to PATH" on Windows).

```
python -m venv .venv
source .venv/bin/activate      # Mac/Linux   |   Windows: .venv\Scripts\activate
pip install -e ".[gui,dev]"
pytest                        # optional proof: 73 tests, ~2 min
python -m fsoc_pat.gui.app     # the live console (the stage demo)
```

Headless run + report:

```
fsoc-pat --headless scenarios/iss_pass.yaml --duration 60 --out runs/
```

Monte-Carlo campaign (regenerates the 0/64 number):

```
python -m fsoc_pat.campaign scenarios/leo_pass_nominal.yaml -n 64 --out runs/mc
```

3. Get the double-clickable app (.exe) — no toolchain needed

GitHub builds it on every push: 1. Open <https://github.com/pranshukesavmishra/SIH-2026/actions> (first time: click "Enable Actions" if asked, then re-run the newest "Build standalone app" run). 2. Open the newest green "Build standalone app" run → bottom → **Artifacts**. 3. Download `fsoc-pat-windows` (stage laptop) or `fsoc-pat-macos` (your Mac). 4. Unzip → double-click `fsoc-pat.exe` (Windows) / `fsoc-pat` (Mac). Carry the zip on a pendrive as backup for the stage.

Build it locally instead (on the OS you are building for):

```
packaging\build.bat          # Windows → dist\fsoc-pat\fsoc-pat.exe
packaging/build.sh           # Mac/Linux → dist/fsoc-pat/fsoc-pat
```

4. Regenerate the telemetry recordings (what the website replays)

Only needed if scenarios or the engine change. From the venv:

```
python tools/export_telemetry.py scenarios/iss_pass.yaml --duration 45 --out docs/media/telemetry_run.json
python tools/export_telemetry.py scenarios/turbulence_hard.yaml --duration 30 --out docs/media/telemetry_turbulence.json
python tools/export_telemetry.py scenarios/decoy_field.yaml --duration 130 --out docs/media/telemetry_decoys.json
```

Rule: if `telemetry_run.json`'s summary changes, the deck's slide-2 chip numbers and chart must be regenerated to match (see §6). Never let the deck and the demo disagree.

5. The website (ZD-1 replay console)

- Root page = `docs/index.html` (identical copy of `docs/demo.html`); old showcase = `docs/about.html`; data = `docs/media/telemetry_*.json`.
- Test locally: `cd docs && python -m http.server 8000` → `http://localhost:8000`
- **Update Netlify**: `app.netlify.com` → site `zerodrift-fsoc-pat` → Deploys → drag-and-drop the whole `docs/` folder.
- 30-second verification after every deploy: tiles read **1.27 s / 98.3 % / 198 μrad / 100 %**; constants line reads **FOV 6° · 30 fps · blink 4 Hz · slew limit 20°/s**; Decoy-field scenario at ~T+90 s shows red DECOY diamonds. If anything differs, the deploy is wrong.

6. Rebuild the pitch deck

The deck is generated code → template-perfect every time. The build scripts live in the Claude session scratchpad (see `PROJECT_MEMORY.md` §Deck build system); the shipped copies are `docs/submission/ZeroDrift_SIH26169.{pdf,pptx}`. For manual edits: open the PPTX in PowerPoint — but re-check every number against §4's recordings and keep the template untouched (headings, footer, 6 slides, ≤10 MB, filename `ZeroDrift_SIH26169.pdf`).

7. Zero-cost physical demo (₹0, optional, never in the deck)

Full guide: `docs/zero_cost_demo.md`. Short version:

```
pip install opencv-python numpy
python tools/webcam_beacon_demo.py --blink 4
```

Phone flashlight strobing at ~4 Hz = green BEACON CONFIRMED; steady lights = red rejected. This is the blink-signature identity principle, live, with hardware you already own.

8. Team preparation

- `docs/team/ZeroDrift_Team_Explainer.pdf` — what the beacon is, where the pixels come from, judge Q&A, stage roles. Everyone reads this twice.
- `docs/pitch_script.md` — the 3-minute pitch. `docs/defence_brief.md` — hard questions. `docs/how_to_win.md` — strategy.
- `PROJECT_MEMORY.md` (repo root) — full project state; paste into any Claude session to continue work.

9. What we deliberately do NOT build

No optical hardware, no motorised rig (`docs/hardware_guide.md` is kept for reference only and is NOT part of the plan or the pitch). The problem statement asks for validation without optical hardware — that is our positioning, not a gap.

10. Submission checklist (internal round)

- ☐ `ZeroDrift_SIH26169.pdf` uploaded to the form (≤ 10 MB, 6 slides)
- ☐ Progress Stage: 5 · Team name: ZeroDrift · Team ID: from portal/SPOC
- ☐ Netlify site verified per §5 after the latest deploy
- ☐ .exe zip downloaded from Actions and tested on the stage laptop
- ☐ One live run rehearsed: leo_pass_nominal, truth OFF → turbulence up → truth ON reveal

ZeroDrift · SIH 2026 · PS 26169 (ISRO) · Jabalpur Engineering College — repo: github.com/pranshukesavmishra/SIH-2026 · demo: zerodrift-fsoc-pat.netlify.app